Assignment1-Doc

October 9, 2025

Name: ___HAN Zhuo___

EID: ___zhuohan3___

1 CS5489 - Assignment 1 - SMS classification

1.1 Goal

In this assignment, the task is predict whether an SMS message is a real message, a spam message, or a phishing message (called smishing). Here are some examples:

- **Normal**: "For real tho this sucks. I can't even cook my whole electricity is out. And I'm hungry."
- **Spam**: "Had your mobile 10 mths? Update to latest Orange camera/video phones for FREE. Save £s with Free texts/weekend calls. Text YES for a callback orno to opt out"
- **Smishing**: "Todays Vodafone numbers ending 5347 are selected to receive a Rs.2,00,000 award. If you have a match please call 6299257179 quoting claim code 2041 standard rates apply"

Your goal is to train a classifier to predict the class from the SMS text.

1.2 Methodology

You need to train classifiers using the training data, and then predict on the test data. You are free to choose the feature extraction method and classifier algorithm. You are free to use methods that were not introduced in class. You should probably do cross-validation to select a good parameters.

1.3 Evaluation

You need to report your test predictions. The csv file has determined the split of validation and test data, where the validation data will be used to determine the timestep of checkpointing. The model that achieves the best performance on validation set should be used to evaluate the test data. The test performance will be used to calculate your final ranking.

The evaluation metric is **balanced accuracy score**. This is because the dataset has some class imbalance as there are more normal samples than spam/smishing samples. See details for `sklearn.metrics.balanced_accuracy_score` here.

1.4 What to hand in

You need to turn in the following things:

1. This ipynb file `Assignment1-Doc.ipynb` with your source code and documentation. ***You should write about all the various attempts that you make to find a good solution.*** You may also submit python scripts as source code, but you then must document all analysis and results (figures, outputs, etc.) in the ipynb file.
2. The PDF file exported from your `Assignment1-Doc.ipynb` file.
3. Your final CSV submission file on test data.
4. The ipynb file `Assignment1-Final.ipynb`, which contains the code that generates the final CSV submission file. **This code will be used to verify that your submission is reproducible.**

Please compress all four files into a single zipfile and upload it to Assignment 1 on Canvas.

1.5 Basic Requirements of Documentation:

For your documentation, you need to at least explain the following things:

- **Data Preprocessing**: This section should detail your exploratory data analysis and the rationale behind all preprocessing steps.
  - Describe the initial characteristics of your dataset.
  - Explain all techniques you have applied on the data
  - Clearly state which subset of the data was used to determine any inherent hyperparameters within your preprocessing techniques, ensuring no information from the test set was used (if any)
- **Methodology**: you will justify your modeling choices.
  - Chosen Models: describe the core principle or model architecture (for deep learning)
  - Chosen Optimizers (if any)
  - Chosen Loss Function (if any)
- **Hyperparameters**: This section should demonstrate a rigorous approach to hyperparameter selection
  - List the key hyperparameters used
  - Document your hyperparameter search process. Compare model performance on a dedicated validation set and select the best-performing configuration based solely on validation metrics. As a core principle of this course, using the test set for hyperparameter selection is strictly prohibited and constitutes academic dishonesty.
- **Results and Visualization**: This section should provide clear evidence of your model's performance and a qualitative analysis of its behavior.
  - Learning Curves (only for deep learning methods): the loss (error) and/or accuracy on training and validation set must be provided.
  - Show examples of correctly classified and misclassified test samples. For misclassified samples, hypothesize why the model failed.

1.6 Grading

The marks of the assignment are distributed as follows: - 45% - Results using various classifiers and feature representations. - 30% - Trying out feature representations (e.g. adding additional features) or classifiers not used in the tutorials/lectures. - 20% - Quality of the written report. More points for insightful observations and analysis. - 5% - Final performance on the test data. If a submission cannot be reproduced by the submitted code, it will not receive marks for ranking. -

Late Penalty: 25 marks will be subtracted for each day late.

NOTE: This is an *individual* assignment.

NOTE: you should start early! Some classifiers may take a while to train.

2 Load the Data

The training data is in the text file `smishing_train.txt`. This CSV file contains the SMS text and the class label. The class labels are: `0`, `1`, `2`, which are `normal`, `spam`, `smishing`.

The validation/testing data is in the text file `smishing_test.txt`, and only contains the SMS text.

The label of validation/testing data is in the csv file `smishing_val_test.csv`, and only contains the SMS text.

You need to generate a csv file, with the following format:

Here are two helpful functions for reading the text data and writing the csv file.

```
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
import matplotlib.pyplot as plt
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
import csv
import pandas as pd
random.seed(100)
```

```
def read_text_data(fname):
    txtdata = []
    classes = []
    with open(fname, 'r', encoding='utf-8') as csvfile:
        reader = csv.reader(csvfile, delimiter=',', quotechar='"')
        for row in reader:
            # get the text
            txtdata.append(row[0])
            # get the class (convert to integer)
            if len(row)>1:
                classes.append(int(row[1]))

    return (txtdata, classes)

def write_csv(fname, Y):
    # fname = file name
    # Y is a list/array with class entries
```

```
    # header
    tmp = [['Id', 'Prediction']]

    # add ID numbers for each Y
    for (i,y) in enumerate(Y):
        tmp2 = [(i+1), y]
        tmp.append(tmp2)

    # write CSV file
    with open(fname, 'w') as f:
        writer = csv.writer(f)
        writer.writerows(tmp)
```

The below code will load the training and test sets.

```
# load the data
(traintxt, trainY) = read_text_data("smishing_train.txt")
(testtxt, _)       = read_text_data("smishing_val_test.txt")
testY              = pd.read_csv("smishing_val_test.csv")
print(len(traintxt))
print(len(testtxt))
testY.head()
```

```
2985
2986
```

[145]:

	Id	Prediction	Usage
0	1	0	val
1	2	0	test
2	3	0	val
3	4	0	test
4	5	0	val

```
# show the classnames
classnames = unique(trainY)
print(classnames)
```

```
[0 1 2]
```

```
classlabels = ['normal', 'spam', 'smishing']
```

Here is an example to write a csv file with predictions on the test set. These are random predictions.

```
# write your predictions on the test set
i = random.randint(len(classnames), size=len(testtxt))
predY = classnames[i]
write_csv("my_submission.csv", predY)
```

Look at the data:

```
for c in classnames:
    tmp = where(trainY==c)
    for a in tmp[0][0:5]:
        print('[{}]: {}'.format(classlabels[trainY[a]], traintxt[a]))
```

```
[normal]: Dunno da next show aft 6 is 850. Toa payoh got 650.
[normal]: I.ll hand her my phone to chat wit u
[normal]: I dont have i shall buy one dear
[normal]: Nite…
[normal]: Ok congrats
[spam]: I'd like to tell you my deepest darkest fantasies. Call me 09094646631
just 60p/min. To stop texts call 08712460324 (nat rate)
[spam]: Santa Calling! Would your little ones like a call from Santa Xmas eve?
Call 09058094583 to book your time.
[spam]: Meet Top 35 US universities in Delhi at India Habitat Centre Lodhi Road
on Nov 8th, 2 to 6 pm for student admission.Entry Free,  details contact
9911489000
[spam]: SMS AUCTION You have won a Nokia 7250i. This is what you get when you
win our FREE auction. To take part send Nokia to 86021 now. HG/Suite342/2Lands
Row/W1JHL 16+
[spam]: Call Germany for only 1 pence per minute! Call from a fixed line via
access number 0844 861 85 86.
[smishing]: WIN URGENT! Your mobile number has been awarded with a £2000 prize
GUARANTEED call 09061790121 from land line. claim 3030 valid 12hrs only 150ppm
[smishing]: Customer service annoncement. You have a New Years delivery waiting
for you. Please call 07046744435 now to arrange delivery
[smishing]: Todays Vodafone numbers ending 9167 are selected to receive a
Rs.2,00,000 award. If you have a match please call 7044518857 quoting claim code
5001 standard rates apply
[smishing]:     Free 1st week entry 2 TEXTPOD 4 a chance 2 win 40GB iPod or £250
cash every wk. Txt VPOD to 81303 Ts&Cs www.textpod.net custcare 08712405020.
[smishing]: 449050000301 You have won a £2,000 price! To claim, call
09050000301.
```

3 YOUR CODE and DOCUMENTATION HERE

4 CS5489 - Assignment 1: SMS Classification

4.1 Sections

0. Reproducibility and Environment
1. Data Preprocessing
2. Methodology and Hyperparameters
3. Evaluation and Analyses

4.1.1 0. Reproducibility and Environment

The following is my end-to-end pipeline for CS5489 assignment 1. A global random seed is set and all experimental settings are reported to ensure reproducibility. All model selection processes are conducted using the validation dataset only.

```
# INSERT YOUR CODE HERE
import os
import sys
import math
import random
import json
import csv
from pathlib import Path

import numpy as np
import pandas as pd
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
```

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
```

```
DATA_DIR = Path(".")
TRAIN_TXT = DATA_DIR / "smishing_train.txt"
VAL_TEST_TXT = DATA_DIR / "smishing_val_test.txt"
VAL_TEST_CSV = DATA_DIR / "smishing_val_test.csv"

for p in [TRAIN_TXT, VAL_TEST_TXT, VAL_TEST_CSV]:
    assert p.exists(), f"File not found: {p}"
```

4.1.2 1. Data Preprocessing

1.1 Dataset Overview Files - `smishing_train.txt`: (content: str, label: int) - `smishing_val_test.txt`: (content: str) - `smishing_val_test.csv`: (id/line_no in txt test file: int, prediction: int, usage: {val, test})

I worked on the three dataset files provided and illustrated the label distributions.

```
(traintxt, trainY) = read_text_data("smishing_train.txt")
(testtxt, _)       = read_text_data("smishing_val_test.txt")
testY              = pd.read_csv("smishing_val_test.csv")
print(len(traintxt), type(traintxt))
```

```
print(len(trainY), type(trainY))
print(len(testtxt), type(testtxt))
testY.tail()
```

```
2985 <class 'list'>
2985 <class 'list'>
2986 <class 'list'>
```

[153]:

	Id	Prediction	Usage
2981	2982	0	test
2982	2983	0	val
2983	2984	0	test
2984	2985	0	val
2985	2986	0	test

```
train_df = pd.DataFrame({'text': traintxt, 'label': trainY})
print(f"Train shape: {train_df.shape}")
# print(train_df.head(3))

valtest_txt_df = pd.DataFrame({"ID": np.arange(1, len(testtxt) + 1, dtype=int),␣
 ↪'text': testtxt})
print(f"Val/Test shape: {valtest_txt_df.shape}")
valtest_txt_df.head(3)
```

```
Train shape: (2985, 2)
Val/Test shape: (2986, 2)
```

[154]:

	ID	text
0	1	Then u drive lor.
1	2	Hey i booked the kb on sat already… what oth…
2	3	K:)k:)good:)study well.

```
for df in (testY, valtest_txt_df):
    df.columns = df.columns.str.strip()
valtest_txt_df.rename(columns={'\ufeffId':'Id','ID':'Id','id':'Id'},␣
 ↪inplace=True)

if 'Id' in testY.index.names: testY = testY.reset_index()
if 'Id' in valtest_txt_df.index.names: valtest_txt_df = valtest_txt_df.
 ↪reset_index()

testY['Id'] = pd.to_numeric(testY['Id'], errors='coerce').astype('Int64')
valtest_txt_df['Id'] = pd.to_numeric(valtest_txt_df['Id'], errors='coerce').
 ↪astype('Int64')

valtest_merged = pd.merge(
    testY, valtest_txt_df,
    on='Id', how='left', validate='one_to_one'
```

```
)
```

```
valtest_merged = pd.merge(
    testY.assign(Id=testY['Id'].astype(int)),
    valtest_txt_df,
    on="Id",
    how="left",
    validate="one_to_one"
    )
print(f"Merged val/test shape: {valtest_merged.shape}")
valtest_merged.head(3)
```

```
Merged val/test shape: (2986, 4)
```

[156]:

	Id	Prediction	Usage	text
0	1	0	val	Then u drive lor.
1	2	0	test	Hey i booked the kb on sat already… what oth…
2	3	0	val	K:)k:)good:)study well.

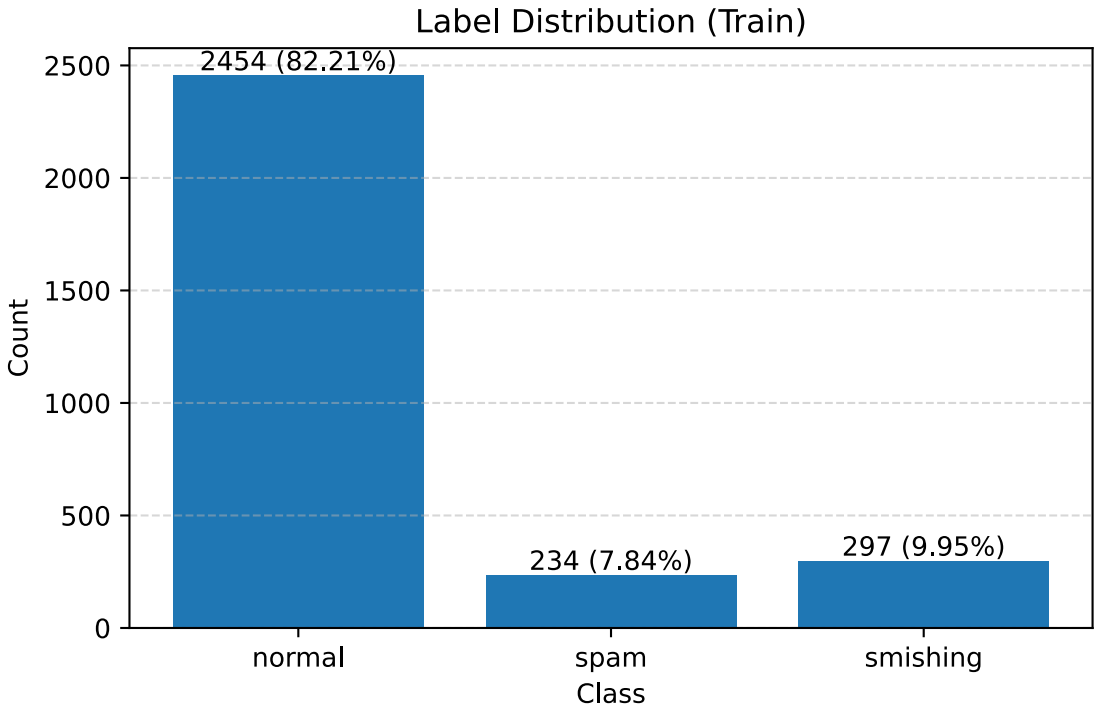
```
if valtest_merged['text'].isnull().any():
    missing = valtest_merged[valtest_merged['text'].isnull()]
    print(f"Missing text for {len(missing)} rows:")
    print(missing)
else:
    print("No missing text entries.")
```

```
No missing text entries.
```

```
labels_names = {0: "normal", 1: "spam", 2: "smishing"}

counts = train_df['label'].value_counts().reindex([0, 1, 2]).fillna(0).
 ↪astype(int)
total = int(counts.sum())
ratios = (counts/max(total)).round(4)

print(f"Label counts:")
for k in [0, 1, 2]:
    print(f"  {labels_names[k]:8s}: {counts[k]:4d} ({ratios[k]:.2%})")
print(f"Total: {total}")

expected = {0: 2454, 1: 234, 2: 297}
mismatch = {k: (counts[k], expected[k]) for k in [0,1,2] if counts[k] !=␣
 ↪expected[k]}
if mismatch:
    print("NOTE: counts differ from assignment sheet (actual, expected) =",␣
 ↪mismatch)
else:
    print("Counts match the assignment sheet.")
```

```
Label counts:
  normal   : 2454 (82.21%)
  spam     :  234 (7.84%)
  smishing:  297 (9.95%)
Total: 2985
Counts match the assignment sheet.
```

```python
names = [labels_names[i] for i in [0, 1, 2]]
vals  = counts.values

plt.figure(figsize=(6, 4))
bars = plt.bar(names, vals)
plt.title("Label Distribution (Train)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.grid(axis="y", linestyle="--", alpha=0.5)

for b, c in zip(bars, vals):
    pct = (c / total * 100.0) if total > 0 else 0.0
    plt.text(b.get_x() + b.get_width()/2, b.get_height(),
             f"{c} ({pct:.2f}%)",
             ha="center", va="bottom", fontsize=10)

plt.tight_layout()
plt.show()
```



4.1.3 1.2 Data Clensing

In this process I first apply NFKC normalization and then replace URLs/emails/numbers in the text with corresponding placeholders. This is to keep the information needed for spam/smishing detection while simplifying the vocabulary.

```
import re, unicodedata, string
from urllib.parse import urlparse

URL_RE = re.compile(r"(https?://\S+|www\.\S+)", re.I)
NUM_RE = re.compile(r"\b\d+(?:[.,]\d+)?\b")
EMAIL_RE = re.compile(r"\b[\w\.-]+@[\w\.-]+\.\w+\b", re.I)

def extract_domain(u: str) -> str:
    try:
        if not u.lower().startswith(("http://", "https://")):
            u = "http://" + u
        host = urlparse(u).hostname or ""
    except Exception:
        host = ""
    return host.lower()

def normalize_trad(s: str) -> str:
    if not isinstance(s, str):
        s = str(s)
    s = unicodedata.normalize("NFKC", s)
    s = URL_RE.sub("__URL__", s)
    s = EMAIL_RE.sub("__EMAIL__", s)
    s = NUM_RE.sub("__NUM__", s)
    s = s.lower()
    s = re.sub(r"\s+", " ", s).strip()
    return s

train_df["text_norm"] = train_df["text"].map(normalize_trad)
valtest_txt_df["text_norm"]   = valtest_txt_df["text"].map(normalize_trad)
valtest_merged["text_norm"]  = valtest_merged["text"].map(normalize_trad)

train_df[["text", "text_norm"]].head(3)
valtest_txt_df[["text", "text_norm"]].head(3)
valtest_merged[["text", "text_norm"]].head(3)

# train_df.head()
# valtest_merged.head()

val_df = (valtest_merged.loc[valtest_merged["Usage"].str.lower()=="val"].
 ↪sort_values("Id").reset_index(drop=True))
```

```
test_df = (valtest_merged.loc[valtest_merged["Usage"].str.lower()=="test"].
 ↪sort_values("Id").reset_index(drop=True))

# val_df  = valtest_txt_df.loc[valtest_merged["Usage"].str.lower()=="val"].
 ↪sort_values("Id").reset_index(drop=True)
# test_df = valtest_txt_df.loc[valtest_merged["Usage"].str.lower()=="test"].
 ↪sort_values("Id").reset_index(drop=True)

print(train_df.shape, val_df.shape, test_df.shape)
# train_df.head(2), val_df.head(2), test_df.head(2)
test_df.head()
```

```
(2985, 3) (1493, 5) (1493, 5)
```

[160]:

	Id	Prediction	Usage	text	\
0	2	0	test	Hey i booked the kb on sat already… what oth…	
1	4	0	test	Hmm .. Bits and pieces lol … *sighs* …	
2	6	0	test	10 min later k…	
3	8	2	test	FREE2DAY sexy St George's Day pic of Jordan!Tx…	
4	10	2	test	Your B4U voucher w/c 27/03 is MARSMS. Log onto…	

	text_norm
0	hey i booked the kb on sat already… what oth…
1	hmm .. bits and pieces lol … *sighs* …
2	__num__ min later k…
3	free2day sexy st george's day pic of jordan!tx…
4	your b4u voucher w/c __num__/__num__ is marsms…

During exploration, we drafted a small set of manual features including `num_urls`, `num_domains`, `brand_in_text`, `tld_com`, `has_money`, and `emoji_cnt`, etc. We **did not** integrate these features into the final pipeline for the following 2 reasons: 1. The examples in the course notes emphasize a clean, comparable approach without these manual engineered features. 2. These features are designed manually and could introduce extra noise in the process. The features selected are what I consider would help the identification. But in the end they are not strictly proved to be the top influences.

```
# NOTE: Archived for documentation only. NOT used by any pipeline in this␣
 ↪submission.
# SHORTENERS = {
#     "t.co","bit.ly","goo.gl","tinyurl.com","ow.ly","buff.ly","t.ly","rebrand.
 ↪ly",
#     "is.gd","s.id","cutt.ly","rb.gy","trib.al","bitly.com","lnkd.
 ↪in","shorturl.at",
#     "zpr.io","forms.gle","youtu.be"
# }
# TOP_TLDS = {"com","net","org","cn","hk","uk","us","info","xyz","io"}

# BRAND_WORDS = {
```

```
#     ␣
 ↪"bank","paypal","apple","amazon","facebook","whatsapp","instagram","google",
#     ␣
 ↪"microsoft","netflix","dhl","fedex","ocbc","dbs","hsbc","citibank","visa","mastercard"
# }

# EMOJI_RE = re.compile(r"[\U0001F300-\U0001FAFF\u2600-\u26FF]+", re.UNICODE)
# URL_FINDER = re.compile(r"(https?://\S+|www\.\S+)", re.I)
```

```
# def brand_in(s: str) -> bool:
#     s_low = s.lower()
#     return any(b in s_low for b in BRAND_WORDS)

# def domain_has_brand(domain: str) -> bool:
#     return any(b in domain for b in BRAND_WORDS)

# def extract_features(text: str) -> dict:
#     raw = str(text)

#     urls = URL_FINDER.findall(raw)
#     domains = [extract_domain(u) for u in urls if extract_domain(u)]
#     tlds = [d.split(".")[-1] for d in domains if "." in d]

#     num_urls = len(urls)
#     num_domains = len(domains)
#     num_short = sum(1 for d in domains if d in SHORTENERS)

#     tld_com = sum(1 for t in tlds if t == "com")
#     tld_cn  = sum(1 for t in tlds if t == "cn")
#     tld_hk  = sum(1 for t in tlds if t == "hk")
#     tld_others = sum(1 for t in tlds if t not in {"com","cn","hk"})

#     brand_text = brand_in(raw)
#     brand_in_domain = any(domain_has_brand(d) for d in domains)
#     brand_mismatch = 1.0 if (brand_text and not brand_in_domain) else 0.0

#     num_6digit_codes = len(re.findall(r"\b\d{6}\b", raw))
#     phone_like = len(re.findall(r"\b\d{8,11}\b", raw))
#     verify_kw = 1.0 if re.search(r"\b(otp|verify|code|verification)\b", raw,␣
 ↪re.I) else 0.0
#     money_hits = re.findall(r"(?:\$|£|€|¥|usd|hkd|sgd|gbp)\s*\d+(?:[.,]\d+)?
 ↪", raw, re.I)
#     has_money = 1.0 if len(money_hits) > 0 else 0.0

#     punct_cnt = sum(ch in string.punctuation for ch in raw)
#     exclaim_cnt = raw.count("!")
#     question_cnt = raw.count("?")
```

```
#     letters = [c for c in raw if c.isalpha()]
#     upper_ratio = (sum(1 for c in letters if c.isupper()) / len(letters)) if␣
 ↪letters else 0.0
#     allcaps_tokens = re.findall(r"\b[A-Z]{3,}\b", raw)
#     alpha_tokens   = re.findall(r"\b[A-Za-z]{3,}\b", raw)
#     allcaps_ratio  = (len(allcaps_tokens) / len(alpha_tokens)) if␣
 ↪alpha_tokens else 0.0

#     emoji_cnt = len(EMOJI_RE.findall(raw))

#     return {
#         "num_urls": num_urls,
#         "num_domains": num_domains,
#         "num_short_urls": num_short,
#         "tld_com": tld_com, "tld_cn": tld_cn, "tld_hk": tld_hk, "tld_others":␣
 ↪tld_others,
#         "brand_in_text": 1.0 if brand_text else 0.0,
#         "brand_in_domain": 1.0 if brand_in_domain else 0.0,
#         "brand_mismatch": brand_mismatch,
#         "num_6digit_codes": num_6digit_codes,
#         "phone_like": phone_like,
#         "verify_kw": verify_kw,
#         "has_money": has_money,
#         "punct_cnt": float(punct_cnt),
#         "exclaim_cnt": float(exclaim_cnt),
#         "question_cnt": float(question_cnt),
#         "upper_ratio": float(upper_ratio),
#         "allcaps_ratio": float(allcaps_ratio),
#         "emoji_cnt": float(emoji_cnt),
#     }

# def build_eng_df(series: pd.Series) -> pd.DataFrame:
#     rows = [extract_features(x) for x in series.tolist()]
#     df = pd.DataFrame(rows)
#     return df[sorted(df.columns)]
```

```
# eng_cols_order = sorted(extract_features("example").keys())

# eng_train = build_eng_df(train_df["text"])
# eng_val   = build_eng_df(val_df["text"])
# eng_test  = build_eng_df(test_df["text"])

# eng_test.head(3)
```

```
# from sklearn.feature_extraction.text import TfidfVectorizer
# from scipy import sparse
# import pickle as pk
# from pathlib import Path

# ART = Path("artifacts_trad")
# ART.mkdir(exist_ok=True)

# # n-gram (1-2)
# tfidf_word = TfidfVectorizer(
#     analyzer="word",
#     token_pattern=r"(?u)\b\w+\b",   #  __URL__ / __NUM__ , etc.
#     ngram_range=(1, 2),
#     min_df=2,
#     max_df=0.95,
#     sublinear_tf=True,
# )

# Xw_tr = tfidf_word.fit_transform(train_df["text_norm"])
# Xw_va = tfidf_word.transform(val_df["text_norm"])
# Xw_te = tfidf_word.transform(test_df["text_norm"])

# # n-gram(3-5)
# tfidf_char = TfidfVectorizer(
#     analyzer="char",
#     ngram_range=(3, 5),
#     min_df=2,
#     sublinear_tf=True,
#     lowercase=False
# )

# Xc_tr = tfidf_char.fit_transform(train_df["text_norm"])
# Xc_va = tfidf_char.transform(val_df["text_norm"])
# Xc_te = tfidf_char.transform(test_df["text_norm"])

# print("word tfidf:", Xw_tr.shape, Xw_va.shape, Xw_te.shape)
# print("char tfidf:", Xc_tr.shape, Xc_va.shape, Xc_te.shape)

# with open(ART / "tfidf_word.pkl", "wb") as f:
#     pk.dump(tfidf_word, f)
# with open(ART / "tfidf_char.pkl", "wb") as f:
#     pk.dump(tfidf_char, f)
```

```
# from scipy.sparse import csr_matrix, hstack
# from sklearn.preprocessing import StandardScaler
# import numpy as np
```

```
# def to_sparse_scaled(df_num: pd.DataFrame, scaler: StandardScaler=None,␣
 ↪fit=False):
#     X = csr_matrix(df_num.astype(np.float32).values)
#     if scaler is None:
#         scaler = StandardScaler(with_mean=False)
#     X_scaled = scaler.fit_transform(X) if fit else scaler.transform(X)
#     return X_scaled, scaler
```

```
# eng_train = eng_train[eng_cols_order]
# eng_val   = eng_val[eng_cols_order]
# eng_test  = eng_test[eng_cols_order]

# Xeng_tr, eng_scaler = to_sparse_scaled(eng_train, fit=True)
# Xeng_va, _ = to_sparse_scaled(eng_val, scaler=eng_scaler, fit=False)
# Xeng_te, _ = to_sparse_scaled(eng_test, scaler=eng_scaler, fit=False)

# X_tr_trad = hstack([Xw_tr, Xc_tr, Xeng_tr], format="csr")
# X_va_trad = hstack([Xw_va, Xc_va, Xeng_va], format="csr")
# X_te_trad = hstack([Xw_te, Xc_te, Xeng_te], format="csr")

# y_train = train_df['label'].to_numpy(np.int64)
# print(f"Final train/val/test shapes: {X_tr_trad.shape}, {X_va_trad.shape},␣
 ↪{X_te_trad.shape}")
```

```
# eng_train.describe().T.assign(is_constant=lambda x: x["std"]==0).head(12)
```

4.1.4 2. Methodology and Hyperparameters

2.1. Task and Metric

- This task is a tri-classification task, where label 0 represents normal, 1 represents spam, and 2 represents smishing.
- Because the classes are imbalanced as illustrated above, we use Macro-F1 for model selection.
- Additionally, we report the Accuracy score and the confusion matrix. #### 2.2. Chosen Models

1. Generative Models
    1. Gaussian Naive bayes (GNB)
    2. Linear Discriminant Analysis (LDA)
    3. Quadratic Discriminant Analysis (QDA)
2. Discriminative Models
    1. Logistic Regression (LR)
    2. Linear SVM
    3. Kernel SVM
3. Others
    1. Decision Tree
    2. Random Forest
    3. MLP All reported pipelines use n-gram TF-IDF (and where appropriate, Truncated

SVD+StandardScaler). Although we implemented a set of manual engineered features (see archived code), these were **not used** in any grid search or final evaluation to preserve a clean comparison and keep the hyper-parameter space manageable. #### 2.3. Search Space Scope Consequently, the hyper-parameter grids reported in this document doesn't involve anything related to manual features (e.g., regex lists, union weights). Selection is solely based on validation Macro-F1 with the models.

```
import numpy as np
import pandas as pd

from sklearn.metrics import accuracy_score, f1_score, confusion_matrix,␣
 ↪classification_report
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, FunctionTransformer
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,␣
 ↪QuadraticDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC, SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier

def evaluate_model(model, name, X_train, y_train, X_val=None, y_val=None):
    model.fit(X_train, y_train)

    # Train
    y_tr = model.predict(X_train)
    acc_tr = accuracy_score(y_train, y_tr)
    f1_tr  = f1_score(y_train, y_tr, average='macro')

    out = {
        "model": name,
        "train_acc": acc_tr,
        "train_macro_f1": f1_tr,
        "val_acc": None,
        "val_macro_f1": None,
        "val_cm": None,
        "val_pred": None,
        "estimator": model
    }

    print(f"\n===== {name} =====")
    print(f"Train Acc: {acc_tr:.4f} | Macro-F1: {f1_tr:.4f}")
```

```
    if X_val is not None:
        y_va = model.predict(X_val)
        out["val_pred"] = y_va
        if y_val is not None:
            acc_va = accuracy_score(y_val, y_va)
            f1_va  = f1_score(y_val, y_va, average='macro')
            cm_va  = confusion_matrix(y_val, y_va)
            out.update({"val_acc": acc_va, "val_macro_f1": f1_va, "val_cm":␣
↪cm_va})
            print(f"Valid Acc: {acc_va:.4f} | Macro-F1: {f1_va:.4f}")
            print("Valid Confusion Matrix (rows=true, cols=pred):\n", cm_va)
        else:
            print("Valid: no labels provided (predictions only).")
    return out
```

```
train_texts = train_df["text_norm"].astype(str).values
train_labels = train_df["label"].astype(np.int64).values
val_texts = val_df["text_norm"].astype(str).values
val_labels = val_df["Prediction"].astype(np.int64).values
```

```
## Generative
### GNB Count n-gram
gnb_count = Pipeline([
    ("count", CountVectorizer(max_features=50000, ngram_range=(1,2))),
    ("todense", FunctionTransformer(lambda X: X.toarray(), accept_sparse=True)),
    ("clf", GaussianNB(var_smoothing=1e-8))
])

### GNB TF-IDF SVD
gnb_tfidf_svd = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=50000, ngram_range=(1,2))),
    ("svd",   TruncatedSVD(n_components=200, random_state=SEED)),
    ("scale", StandardScaler(with_mean=True)),
    ("clf",   GaussianNB(var_smoothing=1e-8))
])

### LDA TF-IDF SVD
lda_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=50000, ngram_range=(1,2))),
    ("svd",   TruncatedSVD(n_components=200, random_state=SEED)),
    ("scale", StandardScaler(with_mean=True)),
    ("clf",   LinearDiscriminantAnalysis(solver="lsqr", shrinkage="auto"))
])

### QDA TF-IDF SVD
qda_pipe = Pipeline([
```

```
    ("tfidf", TfidfVectorizer(max_features=50000, ngram_range=(1,2))),
    ("svd",   TruncatedSVD(n_components=200, random_state=SEED)),
    ("scale", StandardScaler(with_mean=True)),
    ("clf",   QuadraticDiscriminantAnalysis(reg_param=0.1))
])

## Discriminative
### LR
lr_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=50000, ngram_range=(1,2))),
    ("scale", StandardScaler(with_mean=False)),
    ("clf",   LogisticRegression(C=1.0, penalty="l2", solver="liblinear",
                                 class_weight="balanced", max_iter=200,␣
 ↪random_state=SEED))
])

### Linear SVM
linear_svm_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=50000, ngram_range=(1,2))),
    ("scale", StandardScaler(with_mean=False)),
    ("clf",   LinearSVC(C=1.0, class_weight="balanced", random_state=SEED))
])

### Kernel SVM
rbf_svm_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=30000, ngram_range=(1,2))),
    ("scale", StandardScaler(with_mean=False)),
    ("clf",   SVC(kernel="rbf", C=10.0, gamma=1e-4, class_weight="balanced",␣
 ↪random_state=SEED))
])

## Others
### Decision Tree
dt_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=30000, ngram_range=(1,2))),
    ("clf",   DecisionTreeClassifier(max_depth=16, min_samples_leaf=5,␣
 ↪random_state=SEED))
])

### Random Forest
rf_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=30000, ngram_range=(1,2))),
    ("clf",   RandomForestClassifier(n_estimators=400, max_depth=None,␣
 ↪n_jobs=-1, random_state=SEED))
])

### MLP
```

```
mlp_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=50000, ngram_range=(1,2))),
    ("svd",   TruncatedSVD(n_components=200, random_state=SEED)),
    ("scale", StandardScaler(with_mean=True)),
    ("clf",   MLPClassifier(hidden_layer_sizes=(256, 128), activation="relu",
                            alpha=1e-4, learning_rate_init=1e-3,
                            batch_size=64, max_iter=50, early_stopping=True,
                            random_state=SEED))
])
```

```
from sklearn.model_selection import ParameterGrid
import numpy as np
from sklearn.metrics import f1_score, accuracy_score

def fit_and_eval(pipe, Xtr, ytr, Xval, yval):
    pipe.fit(Xtr, ytr)
    val_pred = pipe.predict(Xval)
    return {
        "val_macro_f1": f1_score(yval, val_pred, average="macro"),
        "val_acc": accuracy_score(yval, val_pred),
        "val_pred": val_pred
    }

search_spaces = [
    # Generative
    ("GNB (Count)", gnb_count, {
        "clf__var_smoothing": np.logspace(-12, -7, 6),
    }),
    ("GNB (TF-IDF+SVD)", gnb_tfidf_svd, {
        "clf__var_smoothing": np.logspace(-12, -7, 6),
        "svd__n_components": [100, 200, 300],
    }),

    # LDA/QDA
    ("LDA", lda_pipe, {
        "svd__n_components": [100, 200, 300],
        "clf__shrinkage": ["auto", 0.0, 0.05, 0.1, 0.2, 0.5],
        "clf__solver": ["lsqr"],
    }),
    ("QDA", qda_pipe, {
        "svd__n_components": [100, 200, 300],
        "clf__reg_param": [0.0, 1e-4, 1e-3, 1e-2, 0.05, 0.1, 0.2, 0.5],
    }),

    # Discriminative
    ("LR", lr_pipe, {
        "clf__C": np.logspace(-3, 3, 7),
```

```
        "clf__penalty": ["l2"],
        "clf__solver": ["liblinear"],
        "clf__class_weight": ["balanced"],
        "clf__max_iter": [200, 400],
    }),

    ("Linear SVM", linear_svm_pipe, {
        "clf__C": np.logspace(-3, 3, 7),
        "clf__loss": ["hinge", "squared_hinge"],
        "clf__max_iter": [2000],
    }),

    ("Kernel SVM", rbf_svm_pipe, {
        "tfidf__max_features": [30000, 50000],
        "clf__C": np.logspace(-1, 3, 5),
        "clf__gamma": np.logspace(-4, 0, 5),
    }),

    # Trees
    ("Decision Tree", dt_pipe, {
        "tfidf__max_features": [30000, 50000],
        "clf__max_depth": [5, 10, 20, None],
        "clf__min_samples_leaf": [1, 2, 5, 10],
    }),

    ("Random Forest", rf_pipe, {
        "tfidf__max_features": [30000, 50000],
        "clf__n_estimators": [200, 400, 800],
        "clf__max_depth": [None, 10, 20],
        "clf__min_samples_leaf": [1, 2, 5],
        "clf__n_jobs": [-1],
    }),

    # MLP
    # ("MLP", mlp_pipe, {
    #     "svd__n_components": [100, 200, 300],
    #     "clf__hidden_layer_sizes": [(256, 128), (256,), (384, 128)],
    #     "clf__alpha": np.logspace(-6, -2, 5),         # L2 norm
    #     "clf__learning_rate_init": np.logspace(-4, -2, 3),
    #     "clf__early_stopping": [True],
    #     "clf__max_iter": [80],
    # }),
    ("MLP", mlp_pipe, {
    "tfidf__sublinear_tf": [True],
    "tfidf__min_df": [2],
    "svd__n_components": [200, 300],
    "clf__hidden_layer_sizes": [(512,), (384, 128), (256, 128)],
```

```
    "clf__learning_rate": ["adaptive"],
    "clf__learning_rate_init": [5e-4, 1e-3, 2e-3],
    "clf__alpha": [3e-5, 1e-4, 3e-4],
    "clf__early_stopping": [True],
    "clf__n_iter_no_change": [8, 12],
    "clf__max_iter": [120],
    "clf__batch_size": [64],
}),
]
```

2.4. Validation-Only Search Results We now perform the validation-only grid search. What's printed below is the best setting of each model according to the validation Macro-F1 and corresponding best hyperparameters. Then, we print out the performance on the test set of each model with their best hyperparameters.

```
best_results = []
misclassified_cases = {}
print("===== Hyperparam search on validation (by Macro-F1) =====")
for name, base_pipe, grid in search_spaces:
    best = {"val_macro_f1": -1, "val_acc": -1, "params": None, "val_pred":␣
 ↪None, "model": None}
    for params in ParameterGrid(grid):
        pipe = base_pipe.set_params(**params)
        out = fit_and_eval(pipe, train_texts, train_labels, val_texts,␣
 ↪val_labels)
        if out["val_macro_f1"] > best["val_macro_f1"]:
            best.update({
                "val_macro_f1": out["val_macro_f1"],
                "val_acc": out["val_acc"],
                "params": params,
                "val_pred": out["val_pred"],
                "model": pipe
            })
    print(f"\n[{name}] best on VAL | MaF={best['val_macro_f1']:.4f},␣
 ↪Acc={best['val_acc']:.4f}")
    print("  best params:", best["params"])
    best_results.append((name, best))

print("\n===== Evaluate the chosen settings on TEST =====")
for name, best in best_results:
    best_model = best["model"]
    best_model.fit(train_texts, train_labels)
    test_pred = best_model.predict(test_df["text_norm"])
    current_misclassified_cases = []
    if "Prediction" in test_df.columns:
        X_test = test_df["text_norm"].astype(str).values
        y_test = test_df["Prediction"].astype(np.int64).values
```

```
        test_acc = accuracy_score(test_df["Prediction"], test_pred)
        test_maf = f1_score(test_df["Prediction"], test_pred, average="macro")
        print(f"[{name}] TEST | MaF={test_maf:.4f}, Acc={test_acc:.4f}")

        mask = (test_pred != y_test)
        idxs = np.where(mask)[0]
        for i in idxs:
            row = test_df.iloc[i]
            current_misclassified_cases.append({
                "Id": int(row["Id"]),
                "text": row["text"],
                "true": int(row["Prediction"]),
                "pred": int(test_pred[i])
            })

    else:
        print(f"[{name}] TEST predictions head:", test_pred[:10])
    misclassified_cases.update({name: current_misclassified_cases})
```

```
===== Hyperparam search on validation (by Macro-F1) =====

[GNB (Count)] best on VAL | MaF=0.7997, Acc=0.9203
  best params: {'clf__var_smoothing': np.float64(1e-12)}

[GNB (TF-IDF+SVD)] best on VAL | MaF=0.6963, Acc=0.8513
  best params: {'clf__var_smoothing': np.float64(1e-12), 'svd__n_components':
100}

[LDA] best on VAL | MaF=0.8795, Acc=0.9571
  best params: {'clf__shrinkage': 0.05, 'clf__solver': 'lsqr',
'svd__n_components': 300}

[QDA] best on VAL | MaF=0.8395, Acc=0.9344
  best params: {'clf__reg_param': 0.05, 'svd__n_components': 200}

[LR] best on VAL | MaF=0.8367, Acc=0.9431
  best params: {'clf__C': np.float64(0.01), 'clf__class_weight': 'balanced',
'clf__max_iter': 200, 'clf__penalty': 'l2', 'clf__solver': 'liblinear'}

[Linear SVM] best on VAL | MaF=0.8223, Acc=0.9390
  best params: {'clf__C': np.float64(0.001), 'clf__loss': 'hinge',
'clf__max_iter': 2000}

[Kernel SVM] best on VAL | MaF=0.6233, Acc=0.8788
  best params: {'clf__C': np.float64(1.0), 'clf__gamma': np.float64(0.0001),
'tfidf__max_features': 30000}

[Decision Tree] best on VAL | MaF=0.8093, Acc=0.9350
```

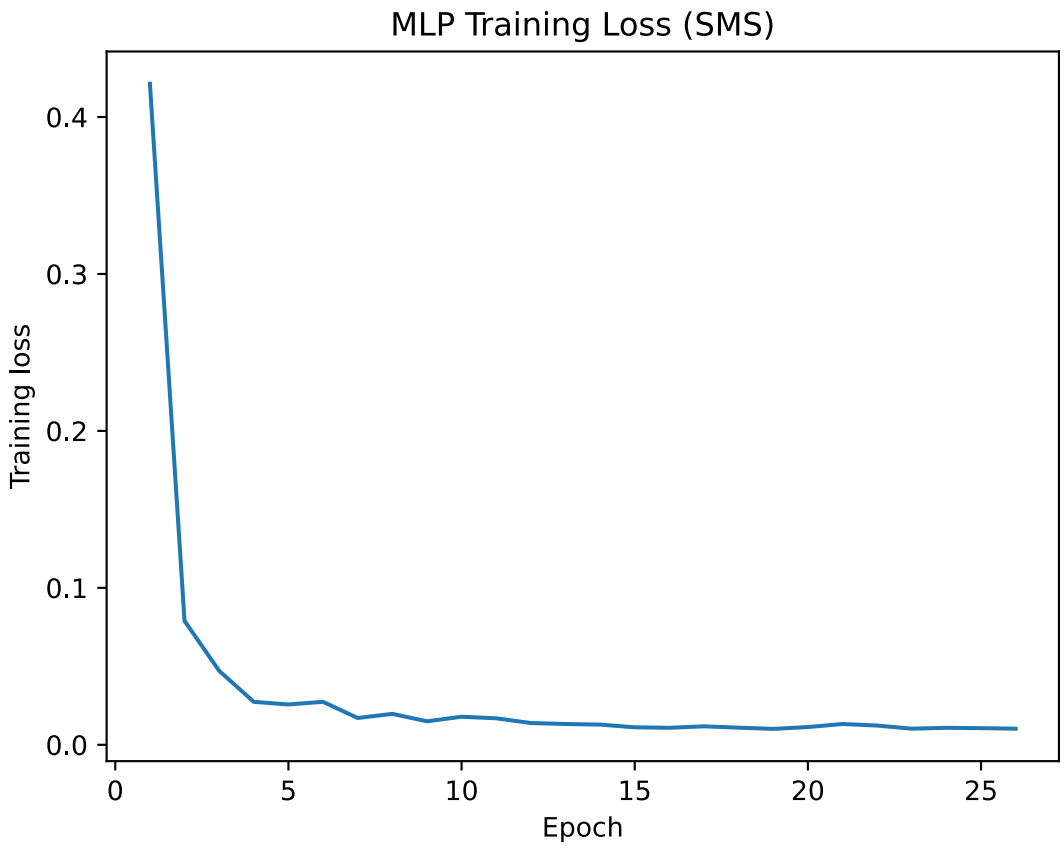
```
  best params: {'clf__max_depth': 20, 'clf__min_samples_leaf': 1,
'tfidf__max_features': 50000}

[Random Forest] best on VAL | MaF=0.8317, Acc=0.9390
  best params: {'clf__max_depth': None, 'clf__min_samples_leaf': 1,
'clf__n_estimators': 200, 'clf__n_jobs': -1, 'tfidf__max_features': 50000}

[MLP] best on VAL | MaF=0.9060, Acc=0.9678
  best params: {'clf__alpha': 0.0003, 'clf__batch_size': 64,
'clf__early_stopping': True, 'clf__hidden_layer_sizes': (256, 128),
'clf__learning_rate': 'adaptive', 'clf__learning_rate_init': 0.0005,
'clf__max_iter': 120, 'clf__n_iter_no_change': 12, 'svd__n_components': 300,
'tfidf__min_df': 2, 'tfidf__sublinear_tf': True}

===== Evaluate the chosen settings on TEST =====
[GNB (Count)] TEST | MaF=0.8269, Acc=0.9344
[GNB (TF-IDF+SVD)] TEST | MaF=0.6603, Acc=0.8473
[LDA] TEST | MaF=0.8632, Acc=0.9491
[QDA] TEST | MaF=0.7168, Acc=0.8299
[LR] TEST | MaF=0.8068, Acc=0.9337
[Linear SVM] TEST | MaF=0.8062, Acc=0.9324
[Kernel SVM] TEST | MaF=0.4824, Acc=0.8346
[Decision Tree] TEST | MaF=0.7616, Acc=0.9169
[Random Forest] TEST | MaF=0.4891, Acc=0.8399
[MLP] TEST | MaF=0.8737, Acc=0.9578
```

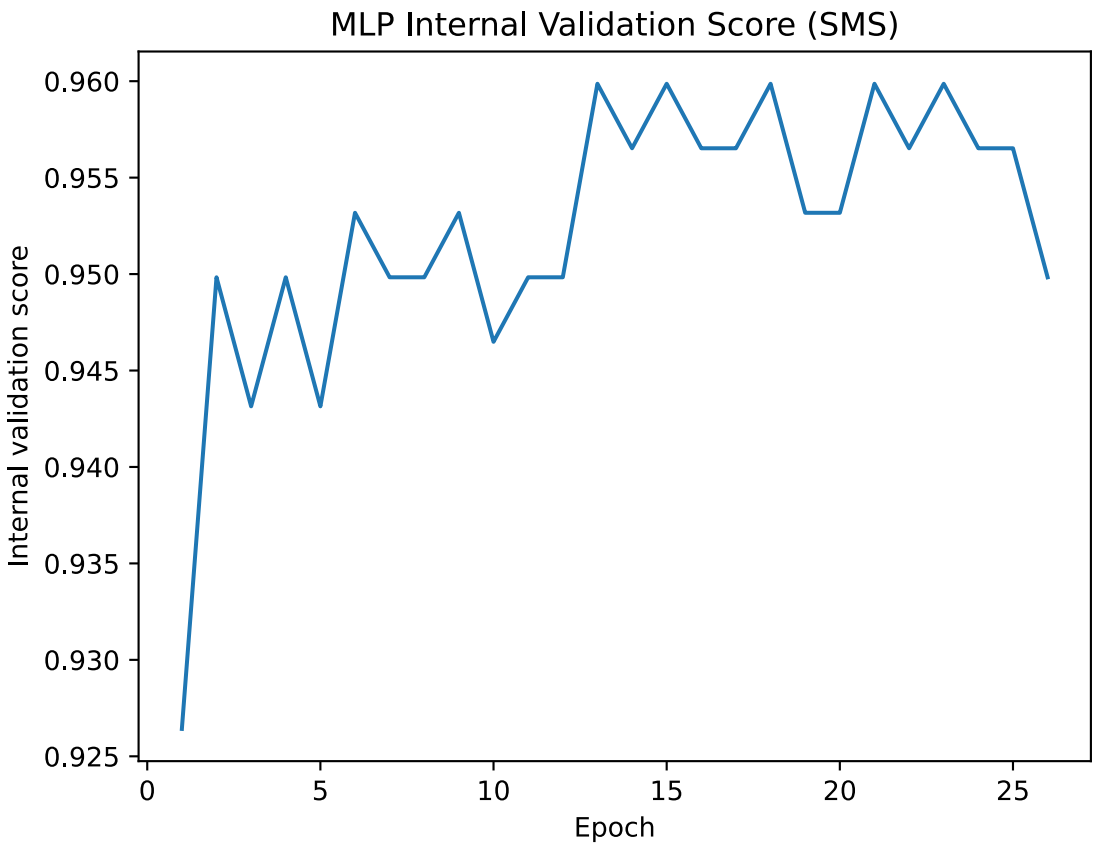
4.1.5 3. Evaluation and Analyses

3.1. The Best Solution: MLP For the MLP, I report epoch-wise training loss and, when available, the internal validation score from scikit-learn's early stopping. These plots verify the convergence.

```python
# Learning Curves for the MLP model
mlp_entry = next(((n, b) for (n, b) in best_results if n.lower().
  ↪startswith("mlp")), None)

if mlp_entry is None:
    print("No MLP in best_results; skip learning curves.")
else:
    name, best = mlp_entry

    mlp_pipe = clone(best["model"])
    mlp_pipe.fit(train_texts, train_labels)


    try:
        mlp = mlp_pipe.named_steps["clf"]
    except KeyError:
```

```
      mlp = next((s for s in mlp_pipe.named_steps.values() if isinstance(s,␣
↪MLPClassifier)), None)

  if mlp is None:
      print("Cannot locate MLPClassifier inside the pipeline.")
  else:

      loss_curve = getattr(mlp, "loss_curve_", None)
      val_scores = getattr(mlp, "validation_scores_", None)

      if loss_curve is not None and len(loss_curve) > 0:
          plt.figure()
          plt.plot(range(1, len(loss_curve) + 1), loss_curve)
          plt.xlabel("Epoch"); plt.ylabel("Training loss")
          plt.title("MLP Training Loss (SMS)")
          plt.show()

      plt.figure()
      plt.plot(range(1, len(val_scores) + 1), val_scores)
      plt.xlabel("Epoch"); plt.ylabel("Internal validation score")
      plt.title("MLP Internal Validation Score (SMS)")
      plt.show()


      yv = mlp_pipe.predict(val_texts)
      print(f"[MLP] External VAL | Macro-F1={f1_score(val_labels, yv,␣
↪average='macro'):.4f}, "
                f"Acc={accuracy_score(val_labels, yv):.4f}")
```


MLP Training Loss (SMS)
Training loss
0.0
0.1
0.2
0.3
0.4
0
5
10
15
20
25
Epoch




```
[MLP] External VAL | Macro-F1=0.9060, Acc=0.9685
```

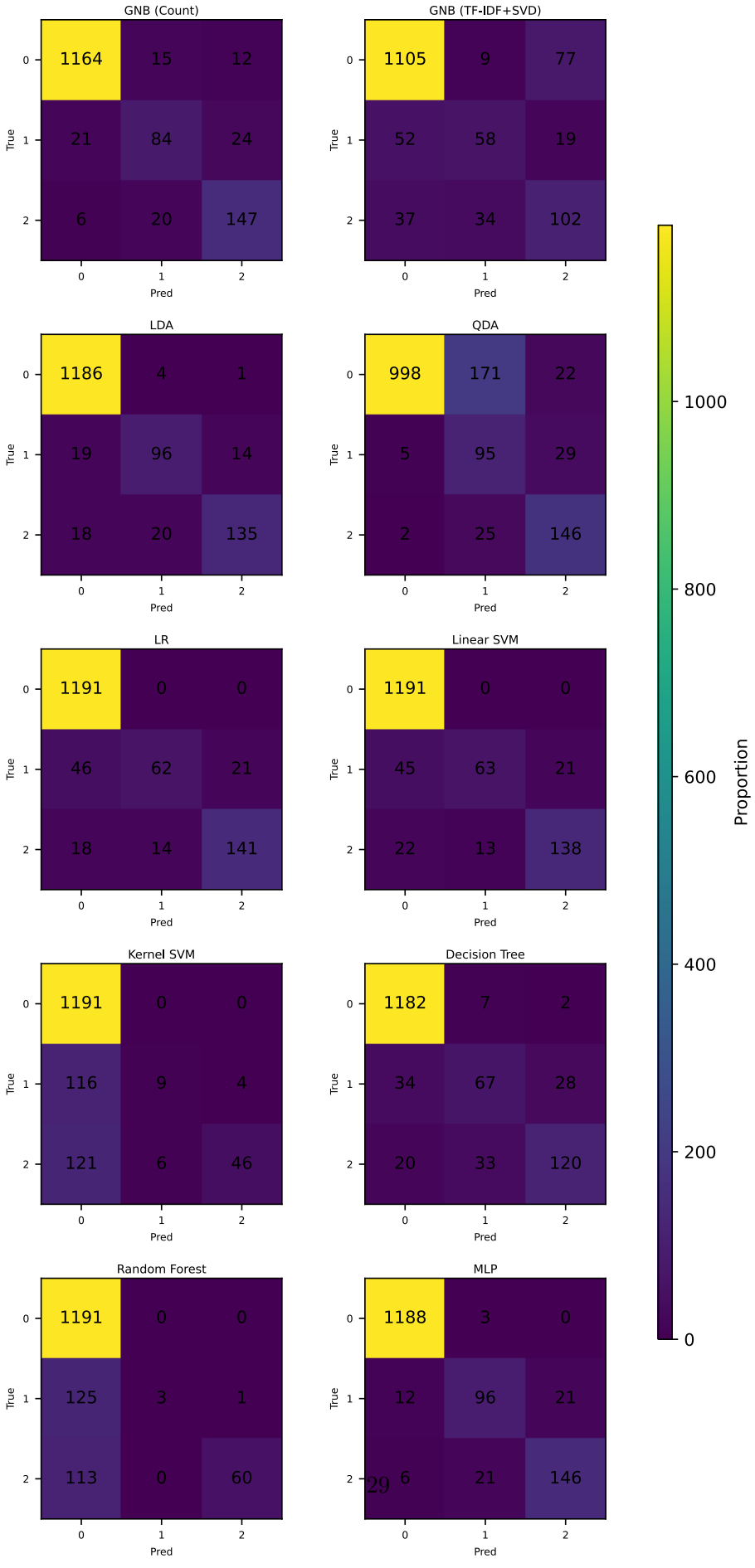
3.2. Confusing Matrix We select the best model on validation dataset in terms of the Macro F1 score. And then we visualize the results using a confusion matrix, where color intensity indicates frequency, and diagonal dominance implies strong performance. From the plot we can observe that most normal messages are correctly classified, while a few spam or smishing texts are occasionally confused with each other.

The figure below displays the confusion matrices of all evaluated models on the test set. Each subplot corresponds to one classifier, allowing for a direct visual comparison of their prediction behaviors across the three SMS categories.

```
X_eval = test_df["text_norm"]
y_eval = test_df["Prediction"]
split_name = "TEST"

COLS = 2            #
cms, names = [], []

for name, info in best_results:
```

```
    pipe = clone(info["model"])
    pipe.fit(train_texts, train_labels)

    y_pred = pipe.predict(X_eval)
    labels_sorted = np.sort(np.unique(np.concatenate([y_eval, y_pred])))
    cm = confusion_matrix(y_eval, y_pred, labels=labels_sorted)
    cms.append(cm)
    names.append(name)

    # print(f"\n=== {name} on {split_name} ===")
    # print(classification_report(y_eval, y_pred, digits=4))

N = len(cms)
rows = math.ceil(N / COLS)

ffig, axes = plt.subplots(rows, COLS, figsize=(2.8 * COLS, 2.6 * rows))
axes = np.array(axes).reshape(rows, COLS)

# import matplotlib.colors as mcolors
# norm = mcolors.LogNorm(vmin=cm.min()+1, vmax=cm.max())

# cmap = cm.get_cmap("magma").copy()
# cmap.set_bad('black')  #   NaN   mask

#   0
# cm_masked = np.ma.masked_less_equal(cm, 0)

for i, (ax, cm, name) in enumerate(zip(axes.ravel(), cms, names)):
    im = ax.imshow(cm, interpolation="nearest", cmap="viridis", vmin=0, vmax=cm.
 ↪max())
    ax.set_title(f"{name}", fontsize=7, pad=3)
    ax.set_xlabel("Pred", fontsize=6)
    ax.set_ylabel("True", fontsize=6)
    labels_sorted = range(cm.shape[0])
    ax.set_xticks(labels_sorted)
    ax.set_yticks(labels_sorted)
    ax.set_xticklabels(labels_sorted, fontsize=6)
    ax.set_yticklabels(labels_sorted, fontsize=6)

    #
    for (r, c), val in np.ndenumerate(cm):
        ax.text(c, r, str(int(val)), ha="center", va="center", fontsize=10)

#
for j in range(i + 1, rows * COLS):
    axes.ravel()[j].axis("off")
```

```
plt.subplots_adjust(wspace=0.35, hspace=0.45, right=0.88)  #      colorbar

#       colorbar
cbar_ax = ffig.add_axes([1.0, 0.15, 0.02, 0.7])  # [left, bottom, width, height]
ffig.colorbar(im, cax=cbar_ax, orientation="vertical", label="Proportion")

plt.tight_layout()
plt.show()
```


GNB (Count)
1164 15 12
21 84 24
6 20 147
GNB (TF-IDF+SVD)
1105 9 77
52 58 19
37 34 102
LDA
1186 4 1
19 96 14
18 20 135
QDA
998 171 22
5 95 29
2 25 146
LR
1191 0 0
46 62 21
18 14 141
Linear SVM
1191 0 0
45 63 21
22 13 138
Kernel SVM
1191 0 0
116 9 4
121 6 46
Decision Tree
1182 7 2
34 67 28
20 33 120
Random Forest
1191 0 0
125 3 1
113 0 60
MLP
1188 3 0
12 96 21
6 21 146
True
Pred
0
1
2
Proportion
1000
800
600
400
200
0

3.3. A Peak at Misclassified Cases By looking at the confusing matrices above and take a brief look at the original texts of misclassified cases of each model, we have the following discussion and analyses.

Observations 1. All models achieve high accuracy for the Normal (0) category, indicated by the bright diagonal in the top-left corner of each matrix. This is expected since the dataset is imbalanced and normal messages dominate the samples. 2. The main performance differences appear in the Spam (1) and Smishing (2) categories, which have fewer examples and more linguistic overlap (e.g., promotional or suspicious content containing URLs and numbers).

Generative Models: Generative models (GNB, LDA, QDA) show more dispersion in the confusion matrices, particularly QDA, which tends to overpredict class 1 (Spam) even for legitimate or smishing messages. This indicates their limited ability to capture complex nonlinear patterns and feature interactions.

Linear Models: Linear models (LR, Linear SVM) produce cleaner diagonals and fewer misclassifications, reflecting their robustness with high-dimensional TF-IDF representations.

Kernel SVM: Kernel SVM (RBF) exhibits improved recall for minority classes but occasionally confuses Spam (1) with Smishing (2), likely due to similar token patterns (e.g., short URLs, monetary cues).

Tree Based Models: Tree-based models (Decision Tree, Random Forest) show clearer separation across classes but sometimes overfit to the majority class, failing to generalize subtle linguistic differences.

MLP: MLP (Feed-forward Neural Network) achieves one of the most balanced confusion patterns, maintaining high precision for Smishing (2) and relatively fewer false positives in Spam (1). The model's capacity to capture nonlinear boundaries explains its stable overall performance.

```
# Take a peak at misclassified examples
# eval_df = pd.DataFrame({
#     "text": X_test,
#     "y_true": y_test,
#     "y_pred": test_pred
# })
# eval_df["correct"] = (eval_df["y_true"] == eval_df["y_pred"])
# N = 10
# print("10 misclassfied examples by MLP:")
# display(eval_df.loc[~eval_df["correct"]][["y_true", "y_pred", "text"]])
for model_name, cases in misclassified_cases.items():
    if not cases:
        print(f"No misclassified cases for {model_name}.")
        continue
    print(f"\n10 misclassified examples by {model_name}:")
    df = pd.DataFrame(cases)
    display(df.head(10))
```

10 misclassified examples by GNB (Count):

	Id	text	true	pred
0	8	FREE2DAY sexy St George's Day pic of Jordan!Tx…	2	1
1	12	thesmszone.com lets you send free anonymous an…	1	0
2	26	U can WIN £100 of Music Gift Vouchers every we…	1	2
3	50	Hey I am really * want to chat or see me text…	2	1
4	130	(Help-Center Canada) You received a Transfer f…	2	0
5	166	Can U get 2 phone NOW? I wanna chat 2 set up m…	1	0
6	238	You are being ripped off! Get your mobile cont…	1	2
7	268	We have sent JD for Customer Service cum Accou…	0	1
8	270	That is wondarfull song	0	1
9	272	\tThe current leading bid is 151. To pause thi…	2	1

10 misclassified examples by GNB (TF-IDF+SVD):

	Id	text	true	pred
0	8	FREE2DAY sexy St George's Day pic of Jordan!Tx…	2	0
1	10	Your B4U voucher w/c 27/03 is MARSMS. Log onto…	2	1
2	12	thesmszone.com lets you send free anonymous an…	1	0
3	30	What class of <#> reunion?	0	2
4	50	Hey I am really * want to chat or see me text…	2	0
5	64	This weeks SavaMob member offers are now acces…	2	0
6	70	Haiyoh… Maybe your hamster was jealous of mi…	0	2
7	90	Claim a 200 shopping spree, just call 08717895…	2	0
8	110	Oh all have to come ah?	0	2
9	130	(Help-Center Canada) You received a Transfer f…	2	0

10 misclassified examples by LDA:

	Id	text	true	pred
0	8	FREE2DAY sexy St George's Day pic of Jordan!Tx…	2	1
1	12	thesmszone.com lets you send free anonymous an…	1	0
2	50	Hey I am really * want to chat or see me text…	2	1
3	130	(Help-Center Canada) You received a Transfer f…	2	0
4	272	\tThe current leading bid is 151. To pause thi…	2	0
5	444	\tI'd like to tell you my deepest darkest fant…	2	1
6	446	We tried to contact you re your reply to our o…	1	2
7	472	Only 1 day left for SAWAN KI BAHAR compt. for …	1	0
8	574	Your K.Y.C has been updated successfully, you …	2	0
9	660	Text & meet someone sexy today. U can find a d…	2	1

10 misclassified examples by QDA:

	Id	text	true	pred
0	4	Hmm .. Bits and pieces lol … *sighs* …	0	1
1	6	10 min later k…	0	1

```
2   8  FREE2DAY sexy St George's Day pic of Jordan!Tx…     2     1
3  26  U can WIN £100 of Music Gift Vouchers every we…     1     2
4  28                          How? Izzit still raining?     0     1
5  30                What class of  &lt;#&gt;  reunion?     0     1
6  34                           Bring tat cd don forget     0     2
7  42                 I wonder if you'll get this text?     0     1
8  50  Hey I am really * want to chat or see me  text…     2     1
9  66  Ok. Can be later showing around 8-8:30 if you …     0     1
```

10 misclassified examples by LR:

```
    Id                                                text  true  pred
0    8  FREE2DAY sexy St George's Day pic of Jordan!Tx…     2     1
1   12  thesmszone.com lets you send free anonymous an…     1     0
2   26  U can WIN £100 of Music Gift Vouchers every we…     1     2
3   50  Hey I am really * want to chat or see me  text…     2     0
4  130  (Help-Center Canada) You received a Transfer f…     2     0
5  166  Can U get 2 phone NOW? I wanna chat 2 set up m…     1     0
6  172  New Offer! Save upto 40\% electricity bill wit…     1     0
7  238  You are being ripped off! Get your mobile cont…     1     2
8  272  \tThe current leading bid is 151. To pause thi…     2     1
9  426  Dear Voucher Holder, 2 claim this weeks offer,…     1     2
```

10 misclassified examples by Linear SVM:

```
    Id                                                text  true  pred
0    8  FREE2DAY sexy St George's Day pic of Jordan!Tx…     2     0
1   12  thesmszone.com lets you send free anonymous an…     1     0
2   26  U can WIN £100 of Music Gift Vouchers every we…     1     2
3   50  Hey I am really * want to chat or see me  text…     2     0
4  130  (Help-Center Canada) You received a Transfer f…     2     0
5  166  Can U get 2 phone NOW? I wanna chat 2 set up m…     1     0
6  172  New Offer! Save upto 40\% electricity bill wit…     1     0
7  238  You are being ripped off! Get your mobile cont…     1     2
8  272  \tThe current leading bid is 151. To pause thi…     2     1
9  426  Dear Voucher Holder, 2 claim this weeks offer,…     1     2
```

10 misclassified examples by Kernel SVM:

```
   Id                                                text  true  pred
0   8  FREE2DAY sexy St George's Day pic of Jordan!Tx…     2     0
1  10  Your B4U voucher w/c 27/03 is MARSMS. Log onto…     2     0
2  12  thesmszone.com lets you send free anonymous an…     1     0
3  26  U can WIN £100 of Music Gift Vouchers every we…     1     0
4  36  ATM BLOCK:Dear customer, Due to our system upg…     2     0
5  50  Hey I am really * want to chat or see me  text…     2     0
6  54  URGENT! Your Mobile number has been awarded wi…     2     0
7  64  This weeks SavaMob member offers are now acces…     2     0
```

```
8  74  Hello from Orange. For 1 month's free access t…     1     0
9  90  Claim a 200 shopping spree, just call 08717895…     2     0
```

10 misclassified examples by Decision Tree:

```
    Id                                                text  true  pred
0   10  Your B4U voucher w/c 27/03 is MARSMS. Log onto…     2     1
1   12  thesmszone.com lets you send free anonymous an…     1     0
2   26  U can WIN £100 of Music Gift Vouchers every we…     1     2
3   46  Yeah so basically any time next week you can g…     0     1
4   50  Hey I am really * want to chat or see me  text…     2     1
5   66  Ok. Can be later showing around 8-8:30 if you …     0     1
6   96  Your 2004 account for 07XXXXXXXXX shows 786 un…     2     1
7  102  URGENT! Your Mobile number has been awarded a …     2     1
8  166  Can U get 2 phone NOW? I wanna chat 2 set up m…     1     2
9  238  You are being ripped off! Get your mobile cont…     1     2
```

10 misclassified examples by Random Forest:

```
   Id                                                text  true  pred
0   8  FREE2DAY sexy St George's Day pic of Jordan!Tx…     2     0
1  10  Your B4U voucher w/c 27/03 is MARSMS. Log onto…     2     0
2  12  thesmszone.com lets you send free anonymous an…     1     0
3  26  U can WIN £100 of Music Gift Vouchers every we…     1     0
4  36  ATM BLOCK:Dear customer, Due to our system upg…     2     0
5  50  Hey I am really * want to chat or see me  text…     2     0
6  64  This weeks SavaMob member offers are now acces…     2     0
7  74  Hello from Orange. For 1 month's free access t…     1     0
8  78   You have 1 new message. Please call 08718738034.     2     0
9  90  Claim a 200 shopping spree, just call 08717895…     2     0
```

10 misclassified examples by MLP:

```
    Id                                                text  true  pred
0    8  FREE2DAY sexy St George's Day pic of Jordan!Tx…     2     1
1   50  Hey I am really * want to chat or see me  text…     2     1
2  166  Can U get 2 phone NOW? I wanna chat 2 set up m…     1     0
3  238  You are being ripped off! Get your mobile cont…     1     2
4  272  \tThe current leading bid is 151. To pause thi…     2     1
5  426  Dear Voucher Holder, 2 claim this weeks offer,…     1     2
6  444  \tI'd like to tell you my deepest darkest fant…     2     1
7  446  We tried to contact you re your reply to our o…     1     2
8  472  Only 1 day left for SAWAN KI BAHAR compt. for …     1     0
9  654  \t5 Free Top Polyphonic Tones call 08701872873…     1     2
```

5 Finally, the output csv file

```
mlp_candidates = [(name, res) for (name, res) in best_results if name.lower().
 ↪startswith("mlp")]
assert len(mlp_candidates) > 0, "best_results    MLP    "

best_mlp_name, best_mlp_info = mlp_candidates[0]
for name, res in mlp_candidates:
    if res["val_macro_f1"] > best_mlp_info["val_macro_f1"]:
        best_mlp_name, best_mlp_info = name, res

print(f"[MLP] best VAL Macro-F1 = {best_mlp_info['val_macro_f1']:.4f}")

best_mlp_pipe = clone(best_mlp_info["model"])
best_mlp_pipe.fit(train_texts, train_labels)

X_test = test_df["text_norm"]
y_test_pred = best_mlp_pipe.predict(X_test)
macro_f1 = f1_score(test_df["Prediction"], y_test_pred, average="macro") if␣
 ↪"Prediction" in test_df.columns else None
acc = accuracy_score(test_df["Prediction"], y_test_pred) if "Prediction" in␣
 ↪test_df.columns else None
print(f"[MLP] TEST result: Macro-F1={macro_f1:.4f}, Acc={acc:.4f}" if macro_f1␣
 ↪and acc else f"[MLP] TEST predictions head: {y_test_pred[:10]}")

output_path = "mysubmission.csv"
write_csv(output_path, y_test_pred)
```

```
[MLP] best VAL Macro-F1 = 0.9060
[MLP] TEST result: Macro-F1=0.8737, Acc=0.9578
```